

RETAILMART V3 • SQL

Reading Query Plans -- Analyst Lens

WhatsApp Announcement

Reading Query Plans (Analyst Lens)

You have been writing SQL for 18 days. Today you learn to READ how PostgreSQL actually EXECUTES your queries. When a query takes 30 seconds instead of 1, EXPLAIN tells you exactly why.

Part 1: EXPLAIN -- See the Plan Before Running

Part 2: EXPLAIN ANALYZE -- See What Actually Happened

Part 3: Common Slow Patterns Analysts Hit

This is not DBA-level optimization. This is the minimum every analyst needs -- knowing how to spot why your query is slow and what to tell your DBA when you need help.

Prerequisites: earlier sessions (all query types). Today applies to every query you have written so far.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: EXPLAIN -- Reading the Plan	2 Hours	What a query plan is, Seq Scan vs Index Scan vs Bitmap Scan, Cost estimates and row estimates
Part 2: EXPLAIN ANALYZE -- Real Execution		Estimated vs actual rows, Actual time measurements, Identifying the bottleneck node
Part 3: Common Analyst Slow Patterns		SELECT * on wide tables, Leading wildcard LIKE, Functions on columns in WHERE, Missing LIMIT on exploration queries

YOUR SQL JOURNEY



SQL Intro



Setup & RetailMart



DDL & Data Types



Constraints & DML



Filtering & Sorting



Scalar Functions



Conditional Logic



Aggregation



Self Join & UNION



Subqueries Part 1



Subqueries & CTEs



Database Concepts



Review & Practice



Window Funcs Part 1



Aggregation & Frames



LAG & LEAD

What We Covered Yesterday

- LAG() to access previous row values
- LEAD() to access next row values
- Period-over-period patterns (MoM, YoY)
- Anomaly detection with percentage thresholds

The 30-Second Query

You write a JOIN between orders and customers to find top spenders. It takes 30 seconds. Your colleague writes a similar query and it finishes in 200ms. The difference? PostgreSQL chose a different execution strategy for each query. EXPLAIN shows you which strategy it picked -- and whether it was a good choice.

What is a Query Plan?

When you run a SQL query, PostgreSQL does NOT execute it top-to-bottom like Python. Instead, it builds an execution plan -- a tree of operations that describes HOW to get your data. EXPLAIN shows you this plan without actually running the query.

EXPLAIN -- Show the Execution Plan

- Shows what PostgreSQL PLANS to do
- Does NOT execute the query (safe on slow queries)
- Shows scan types, join strategies, cost estimates
- Reads bottom-to-top (deepest nodes execute first)

Basic EXPLAIN

```
EXPLAIN  
SELECT * FROM customers.customers  
WHERE email = 'john@gmail.com';
```

Output:

EXPLAIN Output

```
Seq Scan on customers (cost=0.00..1234.00 rows=1 width=120)  
  Filter: (email = 'john@gmail.com'::text)
```

Reading this output:

- Seq Scan -- PostgreSQL will scan EVERY row in the table
- cost=0.00..1234.00 -- estimated startup cost..total cost (arbitrary units)
- rows=1 -- PostgreSQL estimates 1 row will match
- width=120 -- each result row is about 120 bytes wide

Three Main Scan Types

Seq Scan (Sequential Scan): Reads EVERY row in the table, one by one. Like reading a book cover to cover to find one sentence. Fine for small tables or when you need most rows.

Index Scan: Uses an index to jump directly to matching rows. Like using a book's index to find the page. Fast when you need few rows from a large table.

Bitmap Index Scan + Bitmap Heap Scan: Builds a bitmap of matching pages from the index, then reads those pages. A middle ground -- faster than Seq Scan when many rows match, more efficient than Index Scan for moderate result sets.

Seq Scan Example

```
-- Seq Scan: reads ALL 50,000 customers  
EXPLAIN SELECT * FROM customers.customers  
WHERE tier = 'Gold';
```

```
-- Output:
```

```
-- Seq Scan on customers (cost=0.00..1500.00 rows=12500 width=120)
```

```
--   Filter: (tier = 'Gold')
```

```
-- WHY Seq Scan? 5,000 out of 50,000 rows = 10%.
```

```
-- PostgreSQL decided scanning everything is faster
```

```
-- than bouncing between index and table.
```

Index Scan Example

```
-- Index Scan: jumps directly to 1 row
EXPLAIN SELECT * FROM customers.customers
WHERE customer_id = 42;

-- Output:
-- Index Scan using customers_pkey on customers
--   (cost=0.29..8.31 rows=1 width=120)
--   Index Cond: (customer_id = 42)

-- WHY Index Scan? Primary key has an index.
-- 1 row out of 50,000 -- index is perfect.
```

Join Types in Plans

When your query joins tables, the plan shows which join algorithm PostgreSQL chose:

Three Join Algorithms

Nested Loop: For each row in table A, scan table B. Fast when one side is tiny (1-100 rows). Slow when both sides are large.

Hash Join: Build a hash table from the smaller side, probe it with the larger side. The workhorse for most analyst joins. Fast and predictable.

Merge Join: Both sides are sorted, then merged like a zipper. Efficient when both sides are already sorted (e.g., by indexed columns).

Join Plan Example

```
EXPLAIN
SELECT o.order_id, c.first_name
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id;

-- Output:
-- Hash Join  (cost=1500.00..8500.00 rows=150000 width=20)
--   Hash Cond: (o.cust_id = c.customer_id)
--   -> Seq Scan on orders o  (cost=0.00..5000.00 ...)
--   -> Hash
--         -> Seq Scan on customers c  (cost=0.00..1500.00 ...)

-- Read bottom-up: first scan customers into a hash,
-- then scan orders and probe the hash for each row.
```

PRO TIP

PRO TIP

Read plans bottom-to-top, inside-out. The deepest indented node executes first. The topmost node is the final result.
Arrow (-->) shows child operations feeding into parent operations.

Practice 1: Read a Simple Plan

EASY

SCENARIO: You want to understand how PostgreSQL plans to execute a basic filtered query.

TASK: Run EXPLAIN on this query and answer: What scan type does PostgreSQL use? What is the estimated row count? What is the estimated total cost?

HINT: Look for the scan type at the beginning of the output line. The cost and rows are in parentheses.

Query to EXPLAIN

 ANSWER

```
EXPLAIN
SELECT order_id, net_total, order_status
FROM sales.orders
WHERE order_status = 'Delivered';
```

Answer

✓ ANSWER

```
-- Expected output (approximately):
-- Seq Scan on orders (cost=0.00..5500.00 rows=82500 width=20)
--   Filter: (status = 'Delivered')

-- Analysis:
-- Scan type: Seq Scan (sequential -- reads every row)
-- Estimated rows: ~82,500 (55% of 150,000 orders)
-- Total cost: ~5,500 (arbitrary units)
-- WHY Seq Scan? Over half the table matches 'Delivered'.
-- An index would not help -- too many matching rows.
```

Practice 2: Compare Two Join Plans

MEDIUM

SCENARIO: You want to understand how PostgreSQL joins tables differently based on table sizes.

TASK: Run EXPLAIN on both queries below. Compare the join algorithms chosen. Why does PostgreSQL pick different strategies?

HINT: Query A joins all 150K orders to 50K customers. Query B filters to 1 order first, then joins. Look at the join algorithm and scan types in each plan.

Query A -- Large Join



ANSWER

```
EXPLAIN
SELECT o.order_id, c.first_name, c.email
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id;
```

Query B -- Small Join

 ANSWER

```
EXPLAIN
SELECT o.order_id, c.first_name, c.email
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
WHERE o.order_id = 100;
```

Answer



```
-- Query A: Hash Join
--   Scans all of customers into a hash table
--   Scans all of orders and probes the hash
--   Both sides are large -> Hash Join is efficient

-- Query B: Nested Loop
--   Index Scan on orders (1 row: order_id = 100)
--   Index Scan on customers (1 lookup by customer_id)
--   One side is 1 row -> Nested Loop is perfect

-- Key insight: PostgreSQL picks the join algorithm
-- based on how many rows each side produces.
-- Small + Large = Nested Loop.
-- Large + Large = Hash Join.
```

Estimates vs Reality

EXPLAIN shows what PostgreSQL PLANS to do based on table statistics. But statistics can be wrong -- maybe the table grew, or data distribution changed. EXPLAIN ANALYZE actually RUNS the query and shows you what REALLY happened: actual row counts, actual time, actual loops.

The Difference

```
-- EXPLAIN: estimates only, does NOT run the query
EXPLAIN SELECT ...;

-- EXPLAIN ANALYZE: runs the query AND shows actuals
EXPLAIN ANALYZE SELECT ...;

-- EXPLAIN (ANALYZE, BUFFERS): also shows I/O details
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
```

EXPLAIN ANALYZE Safety Warning

EXPLAIN ANALYZE actually executes the query: Unlike plain EXPLAIN, ANALYZE runs the query for real. On a slow query, this means waiting for it to finish.

Never use EXPLAIN ANALYZE on INSERT/UPDATE/DELETE without wrapping in a transaction: It will execute the mutation! Wrap in BEGIN ... ROLLBACK to see the plan without changing data.

EXPLAIN ANALYZE Example

```
EXPLAIN ANALYZE
SELECT c.first_name, COUNT(*) AS order_count
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
GROUP BY c.first_name
ORDER BY order_count DESC
LIMIT 10;
```

EXPLAIN ANALYZE Output

```
Limit  (cost=12000..12001 rows=10 width=15)
      (actual time=450.2..450.3 rows=10 loops=1)
    -> Sort  (cost=12000..12050 rows=5000 width=15)
          (actual time=450.1..450.2 rows=10 loops=1)
        Sort Key: count(*) DESC
        Sort Method: top-N heapsort  Memory: 25kB
    -> HashAggregate  (cost=11000..11500 rows=5000 ...)
          (actual time=420.5..440.1 rows=4800 ...)
    -> Hash Join  (cost=1500..8500 rows=150000 ...)
          (actual time=50.2..280.5 rows=150000 ...)
        Hash Cond: (o.cust_id = c.customer_id)
        -> Seq Scan on orders o
              (actual time=0.01..80.5 rows=150000 loops=1)
        -> Hash
              -> Seq Scan on customers c
                    (actual time=0.01..25.3 rows=50000 loops=1)
Planning Time: 0.5 ms
Execution Time: 451.0 ms
```

EXPLAIN ANALYZE Fields (1/2)

actual time=X..Y: X = time to first row, Y = time to last row (milliseconds). The difference between start and end is how long this node took.

rows=N: Actual number of rows this node produced. Compare to estimated rows -- big mismatches signal stale statistics.

loops=N: How many times this node executed. In nested loops, the inner side runs once per outer row. Total time = actual time * loops.

Planning Time: Time PostgreSQL spent building the plan. Usually < 1ms.

Execution Time:

EXPLAIN ANALYZE Fields (2/2)

Total wall-clock time to run the query. This is what you want to minimize.

Spotting the Bottleneck

The bottleneck is the node with the highest actual time. In the example above:

- Hash Join: 280ms -- this is where most time was spent
- Seq Scan on orders: 80ms -- scanning 150K rows
- HashAggregate: 160ms -- grouping 150K rows into 5K
- Sort + Limit: 10ms -- fast (only top 10)

The bottleneck is the Hash Join + its child scans. To speed this up, you would need an index on the join column or reduce the number of rows entering the join.

PRO TIP

PRO TIP

When estimated rows and actual rows differ by more than 10x, PostgreSQL chose the wrong plan. Run ANALYZE on the table to update statistics: `ANALYZE customers.customers;`

Practice 3: Find the Bottleneck

MEDIUM

SCENARIO: A dashboard query is taking 5 seconds. You need to identify which part is slow.

TASK: Run EXPLAIN ANALYZE on the query below. Identify: (1) the total execution time, (2) which node takes the most time, (3) whether estimated vs actual rows differ significantly.

HINT: Look at each node's actual time. The node with the highest time (and its children) is the bottleneck. Check if rows estimates match actuals.

Slow Query

✓ ANSWER

```
EXPLAIN ANALYZE
SELECT
    r.region_name,
    cat.category_name,
    SUM(oi.quantity * oi.unit_price) AS revenue
FROM sales.order_items oi
JOIN sales.orders o ON oi.order_id = o.order_id
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region r ON s.region_id = r.region_id
JOIN products.products p ON oi.prod_id = p.product_id
JOIN core.dim_brand b ON p.brand_id = b.brand_id
JOIN core.dim_category cat
    ON b.category_id = cat.category_id
WHERE o.order_status = 'Delivered'
GROUP BY r.region_name, cat.category_name
ORDER BY revenue DESC;
```

Answer



```
-- Run the query and read the plan.  
-- Likely bottleneck: the Seq Scan on order_items  
-- (largest table, ~350K rows).  
  
-- Key observations:  
-- 1. Total Execution Time: shown at bottom  
-- 2. Bottleneck: the node with highest actual time  
--    (probably the hash join between orders and order_items)  
-- 3. Row estimates: compare 'rows=X' in estimates  
--    vs 'rows=Y' in actuals. If 10x+ off, run ANALYZE.  
  
-- This is a 4-table join with aggregation.  
-- No single 'fix' -- just understanding WHERE the time goes.
```

The Four Sins of Analyst SQL

DBAs see the same analyst mistakes over and over. These four patterns turn a 100ms query into a 10-second query. Each one is easy to fix once you know what to look for in the plan.

SELECT * Problem

-- BAD: fetches ALL 40+ columns when you need 3

EXPLAIN ANALYZE

SELECT * FROM sales.orders

WHERE cust_id = 42;

-- width=320 (fetching 320 bytes per row)

-- GOOD: fetch only what you need

EXPLAIN ANALYZE

SELECT order_id, order_date, net_total

FROM sales.orders

WHERE cust_id = 42;

-- width=24 (fetching 24 bytes per row)

-- Why it matters: more data = more disk I/O,

-- more memory for sorts, more network transfer.

-- On a 150K row table, the difference is huge.

Leading Wildcard Problem

```
-- BAD: '%gmail.com' -- cannot use an index
EXPLAIN
SELECT * FROM customers.customers
WHERE email LIKE '%gmail.com';
-- Seq Scan (reads ALL 50,000 rows)

-- GOOD: 'john%' -- can use a B-tree index
EXPLAIN
SELECT * FROM customers.customers
WHERE email LIKE 'john%';
-- Index Scan (jumps to 'john' in the index)

-- WHY? B-tree indexes are sorted left-to-right.
-- 'john%' = 'find everything starting with john'
-- '%gmail' = 'the ending could be anywhere' -> full scan
```

Function-on-Column Problem

```
-- BAD: UPPER() wraps the column -- index cannot be used
EXPLAIN
SELECT * FROM customers.customers
WHERE UPPER(email) = 'JOHN@GMAIL.COM';
-- Seq Scan (the index is on 'email', not 'UPPER(email)')

-- GOOD: use ILIKE or match the stored case
EXPLAIN
SELECT * FROM customers.customers
WHERE email ILIKE 'john@gmail.com';
-- Can use index with pg_trgm extension

-- OR: create an expression index (Indexing)
-- CREATE INDEX idx_email_upper
--   ON customers.customers (UPPER(email));
-- Then UPPER(email) = 'JOHN@GMAIL.COM' uses the index.
```

This is called 'sargability' -- a predicate is sargable (Search ARGument ABLE) if the database can use an index. Wrapping a column in a function makes it non-sargable.

Missing LIMIT Problem

```
-- BAD: exploring a table without LIMIT
SELECT * FROM web_events.page_views
ORDER BY view_timestamp DESC;
-- Sorts ALL 500,000 rows, returns ALL of them
-- Your tool only shows the first 100 anyway!

-- GOOD: add LIMIT for exploration
SELECT * FROM web_events.page_views
ORDER BY view_timestamp DESC
LIMIT 100;
-- Sorts only enough to find top 100
-- (top-N heapsort -- much faster than full sort)
```

PRO TIP

A quick sargability checklist: avoid WHERE UPPER(col), WHERE EXTRACT(... FROM col), WHERE col + 1 = X. Instead: WHERE col ILIKE, WHERE col >= date, WHERE col = X - 1. Keep the column bare on one side.

Reading Plans for Rewrites

Sometimes rewriting a query changes the plan entirely:

OR vs UNION ALL

```
-- SLOW: OR prevents index usage
EXPLAIN ANALYZE
SELECT * FROM sales.orders
WHERE order_status = 'Delivered' OR order_status = 'Shipped';
-- Often: Seq Scan with a combined filter

-- FASTER: UNION ALL lets each branch use an index
EXPLAIN ANALYZE
SELECT * FROM sales.orders WHERE order_status = 'Delivered'
UNION ALL
SELECT * FROM sales.orders WHERE order_status = 'Shipped';
-- Can do: two Index Scans, appended together

-- NOTE: IN(...) is usually equivalent to OR
-- but PostgreSQL optimizes IN better than OR in many cases.
```

Practice 4: Rewrite for Sargability

HARD

SCENARIO: A team member wrote a query to find all orders from January 2025. It is slow because it uses EXTRACT on the indexed order_date column.

TASK: Rewrite the query below to be sargable (index-friendly). Run EXPLAIN on both versions and compare the scan types.

HINT: Replace the EXTRACT calls with a simple date range using `>=` and `<`. This keeps order_date bare and allows index usage.

Slow Query

✓ ANSWER

```
-- Non-sargable (function on column):  
EXPLAIN  
SELECT order_id, order_date, net_total  
FROM sales.orders  
WHERE EXTRACT(YEAR FROM order_date) = 2025  
      AND EXTRACT(MONTH FROM order_date) = 1;
```

Answer

✓ ANSWER

```
-- Sargable rewrite (bare column):  
EXPLAIN  
SELECT order_id, order_date, net_total  
FROM sales.orders  
WHERE order_date >= '2025-01-01'  
      AND order_date < '2025-02-01';  
  
-- Original: Seq Scan (EXTRACT wraps the column)  
-- Rewrite: Index Scan (if index exists on order_date)  
  
-- Key lesson: date range with >= and < is ALWAYS  
-- better than EXTRACT for filtering.  
-- Works for any period: year, month, week, day.
```

Practice 5: Plan Before and After UNION ALL

HARD

SCENARIO: You have a query using OR that is slower than expected. Your DBA suggests rewriting it as UNION ALL.

TASK: Run EXPLAIN on both versions below. Compare: (1) scan types used, (2) estimated cost, (3) whether an index is used in either version.

HINT: If there is an index on status, the UNION ALL version may use two Index Scans instead of one Seq Scan. If there is no index, both may use Seq Scan and the OR version might be faster (one scan vs two). The answer depends on your data.

Version A -- OR



ANSWER

EXPLAIN

```
SELECT order_id, order_date, order_status  
FROM sales.orders  
WHERE order_status = 'Cancelled'  
       OR order_status = 'Returned';
```

Version B -- UNION ALL

 ANSWER

```
EXPLAIN
SELECT order_id, order_date, order_status
FROM sales.orders
WHERE order_status = 'Cancelled'
UNION ALL
SELECT order_id, order_date, order_status
FROM sales.orders
WHERE order_status = 'Returned';
```

Answer

✓ ANSWER

```
-- Run both and compare.
-- Likely outcomes:

-- If NO index on status:
--   Both use Seq Scan. OR version is faster
--   (one pass vs two passes over the table).

-- If index on status EXISTS:
--   OR: might still Seq Scan (optimizer merges conditions)
--   UNION ALL: two Bitmap Index Scans, then Append
--   UNION ALL can be faster with index support.

-- Also try: WHERE status IN ('Cancelled', 'Returned')
-- PostgreSQL often optimizes IN as well as UNION ALL.
```

Q: What is the difference between EXPLAIN and EXPLAIN ANALYZE?

A: EXPLAIN shows the estimated execution plan without running the query -- safe and instant. EXPLAIN ANALYZE actually runs the query and shows both estimates and actual measurements (time, rows, loops). Use EXPLAIN for quick plan checks; use EXPLAIN ANALYZE when you need to see real performance numbers. Never run EXPLAIN ANALYZE on mutations without wrapping in BEGIN/ROLLBACK.

Q: What does a Seq Scan mean and when is it a problem?

A: Seq Scan means PostgreSQL reads every row in the table sequentially. It is NOT always bad -- for small tables or queries that need most rows, Seq Scan is optimal. It becomes a problem when you filter a large table down to a few rows (e.g., 10 out of 1 million) and a Seq Scan reads all 1 million. In that case, an Index Scan would be faster, and the fix is usually adding an index.

Q: What makes a WHERE clause non-sargable?

A: A WHERE clause is non-sargable when you wrap the column in a function: WHERE UPPER(email) = ..., WHERE EXTRACT(YEAR FROM date) = ..., WHERE col + 1 = X. The index is built on the raw column value, not on the function output. Fix by keeping the column bare: WHERE email ILIKE ..., WHERE date >= ... AND date < ..., WHERE col = X - 1.

HW 1: Plan Vocabulary

EASY

SCENARIO: Your team lead asks you to explain a query plan during a code review.

TASK: Run EXPLAIN on the following query. In your own words, describe what each line means: the scan type, the filter, the cost, and the estimated rows.

HINT: Describe each field in the output: Seq Scan vs Index Scan, Filter, cost=start..total, rows=N, width=N.

Query

✓ ANSWER

```
EXPLAIN
SELECT product_id, product_name, price
FROM products.products
WHERE price > 50000
      AND cost_price < 30000;
```

Answer

✓ ANSWER

```
-- Expected output (approximately):
-- Seq Scan on products  (cost=0.00..45.00 rows=50 width=40)
--   Filter: ((category = 'Electronics') AND (price > 50000))

-- Explanation:
-- Seq Scan: reads all ~1,500 products row by row
-- Filter: checks each row against both conditions
-- cost=0.00..45.00: startup=0, total estimated=45 units
-- rows=50: PostgreSQL estimates 50 rows will match
-- width=40: each result row is about 40 bytes
```

HW 2: Estimated vs Actual

MEDIUM

SCENARIO: A query seems slow, and you suspect PostgreSQL is making bad estimates.

TASK: Run EXPLAIN ANALYZE on the query below. Compare estimated rows vs actual rows for each node. If they differ by more than 5x, note which node has the mismatch.

HINT: Look at (rows=estimated) vs (rows=actual) on each line. If the mismatch is large, the optimizer may have chosen a suboptimal plan. Fix: run ANALYZE customers.customers to update statistics.

Query



ANSWER

```
EXPLAIN ANALYZE
SELECT a.city, COUNT(*) AS order_count
FROM sales.orders o
JOIN customers.addresses a
  ON o.cust_id = a.customer_id
WHERE a.city = 'Bengaluru'
GROUP BY a.city;
```

Answer



```
-- Run and compare estimated vs actual rows at each node.  
-- Common finding: the Seq Scan on customers may  
-- estimate fewer rows for city='Bengaluru' than actual,  
-- especially if statistics are stale.  
  
-- If estimated=100 but actual=5000, that is a 50x mismatch.  
-- Fix: ANALYZE customers.customers;  
-- Then re-run EXPLAIN ANALYZE to see improved estimates.
```

HW 3: SELECT * vs Named Columns

MEDIUM

SCENARIO: You want to demonstrate to a junior analyst why SELECT * is wasteful.

TASK: Run EXPLAIN ANALYZE on both queries below. Compare: (1) the width in bytes, (2) execution time, (3) any difference in scan strategy.

HINT: The width field shows bytes per row. Multiply by estimated rows to see total data transfer. Less data = faster query.

Query A

 ANSWER

```
EXPLAIN ANALYZE  
SELECT * FROM sales.orders  
WHERE store_id = 5;
```

Query B

 ANSWER

```
EXPLAIN ANALYZE
SELECT order_id, order_date, net_total
FROM sales.orders
WHERE store_id = 5;
```

Answer

✓ ANSWER

```
-- Query A: width=~320 (all columns)
-- Query B: width=~24 (3 columns)

-- For ~750 rows (1 store out of 200):
-- Query A transfers: 750 * 320 = 240 KB
-- Query B transfers: 750 * 24 = 18 KB
-- 13x less data!

-- Execution time difference may be small on this table,
-- but on large tables the I/O savings are significant.
```

HW 4: Fix the Non-Sargable Query

HARD

SCENARIO: An analyst wrote three queries with performance problems. All use functions on indexed columns, making them non-sargable.

TASK: Rewrite each query to be sargable. Run EXPLAIN on the original and rewrite to verify the scan type changes.

HINT: Move the function/arithmetic to the other side of the comparison. Keep the column bare.

Query A

 ANSWER

```
SELECT * FROM sales.orders
WHERE EXTRACT(YEAR FROM order_date) = 2025;
```

Query B

 ANSWER

```
SELECT * FROM customers.customers  
WHERE UPPER(email) = 'JOHN@GMAIL.COM';
```

Query C



ANSWER

```
SELECT * FROM products.products  
WHERE price + 500 > 10000;
```

Answer

✓ ANSWER

```
-- Query A rewrite:
SELECT * FROM sales.orders
WHERE order_date >= '2025-01-01'
      AND order_date < '2026-01-01';

-- Query B rewrite:
SELECT * FROM customers.customers
WHERE email ILIKE 'john@gmail.com';

-- Query C rewrite:
SELECT * FROM products.products
WHERE price > 9500;  -- moved the +500 to the other side
```

HW 5: Full Plan Analysis Report

CRAZY

SCENARIO: Your manager asks you to analyze a slow dashboard query and write a brief report explaining where the time goes.

TASK: Run EXPLAIN (ANALYZE, BUFFERS) on the query below. Write a 5-line analysis: (1) total execution time, (2) the slowest node, (3) estimated vs actual row accuracy, (4) join algorithm used, (5) one suggestion to speed it up.

HINT: BUFFERS shows disk I/O (shared hit = from cache, shared read = from disk). The node with the most shared read is doing the most disk I/O -- that is your I/O bottleneck.

Query

✓ ANSWER

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    dr.region_name,
    DATE_TRUNC('month', o.order_date) AS month,
    COUNT(DISTINCT o.cust_id) AS unique_customers,
    SUM(oi.quantity * oi.unit_price) AS revenue
FROM sales.order_items oi
JOIN sales.orders o ON oi.order_id = o.order_id
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
WHERE o.order_status = 'Delivered'
GROUP BY dr.region_name, DATE_TRUNC('month', o.order_date)
ORDER BY month, revenue DESC;
```

Answer

✓ ANSWER

```
-- Run the query with (ANALYZE, BUFFERS).
-- Sample analysis:

-- 1. Total Execution Time: ~X ms (from bottom of output)
-- 2. Slowest node: likely the Hash Join between
--    order_items and orders (largest tables)
-- 3. Row accuracy: check estimated vs actual at each node
-- 4. Join algorithm: probably Hash Join for the large joins
--    and Nested Loop for the small stores lookup
-- 5. Suggestion: an index on orders(status) could help
--    the WHERE filter, reducing rows before the join.
--    Or: add WHERE order_date >= ... to limit the range.
```

KEY TAKEAWAYS (1/2)

EXPLAIN shows the plan, EXPLAIN ANALYZE runs it: Use EXPLAIN for quick checks. Use ANALYZE when you need real timing data.

Read plans bottom-up: Deepest indented nodes execute first. The topmost node is the final result.

Seq Scan is not always bad: On small tables or when you need most rows, Seq Scan is optimal.

Big row estimate mismatches mean wrong plans: When estimated rows differ from actual rows by 10x+, run ANALYZE on the table.

Keep WHERE columns bare for sargability:

KEY TAKEAWAYS (2/2)

No functions wrapping indexed columns. Move math to the other side.

SELECT only what you need: Naming columns instead of * reduces width, I/O, and memory usage.

What is Next

Tomorrow you will learn to CREATE indexes -- the primary tool for making slow queries fast. You will understand B-Tree indexes, composite indexes, partial indexes, and when indexing helps vs when it does not.